Solving Panex: the search, its efficiency, and the pattern behind the move counts

A white paper on ShapeLogic's Panex Solver and what its eight optimal solves of the tower swap (levels 3 to 10) reveal

30 October 2026 · Data: `tmp/Panex Swap 3–10.xmpuzzle`, the solver source `core/shapelogic/solve/panex_search.cpp`, level logs from `shapelogic_cli --panex-log`, and exhaustive searches of the small levels

Summary

- **All eight solves are optimal.** The swaps of towers of 3 to 10 pieces take 42, 128, 343, 881, 2,189, 5,359, 13,023 and 31,537 transfers. These are the published minima: Manasse, Sleator and Wei's 1983 algorithm gives these numbers as an upper bound, and exhaustive searches (the authors to level 6, Bagley to level 9, Kisman in 2010 to level 13) proved nothing shorter exists.
- **The counts follow a formula.** From level 5 on, $X(n) = 7\cdot(5P_{n-1} - P_{n-2}) - 8n - 2$ with the Pell numbers P (0, 1, 2, 5, 12, 29, …); equivalently $X(n) = 2X(n-1) + X(n-2) + 16n - 28$. Each extra level multiplies the length by $1 + \sqrt{2} \approx 2.414$ in the limit, the silver ratio. The formula reproduces all nine proven values from level 5 to 13 and predicts 1,074,001 moves for level 14, 2,593,021 for 15 and 6,260,271 for 16.
- **Every solution has the same skeleton.** It begins with a shortest transfer of the n–1 upper pieces to the centre ($T(n-1)$ moves, exactly the published transfer optimum), ends with the mirror image of such a transfer, and at its midpoint both deepest pieces sit at the bottom of the centre column. The small pieces do almost all the moving: sizes 1 to 3 make 74% of the moves at level 10.
- **The work grows 7.8 times faster than the answer.** The solver visits about 18.8 times more stackings for each extra level (1.72 trillion for level 10, in 58.6 hours) while the solution only grows 2.4 times; the widest search level grows 7.6 times. Level 11 would need about 16 GB for a single level and weeks of search: beyond this machine with the current design.
- **Shortest solutions live in a narrow corridor.** For level 5 the state space holds 1.9 million stackings and 6.3 trillion distinct shortest solutions, yet at most 28 states at any distance lie on a shortest solution, against 8,890 states at the middle distance. A search that knew the corridor would do about 300 times less work at level 5, and far more at level 10. Kisman's structural search (30 minutes and 50 MB for level 10) shows that this gain is real.
- **What this means for the solver.** For the standard swap and transfer the answer is known: the solution can be constructed from the published algorithm in milliseconds and is proven optimal to level 13, so a search is only needed as a proof or for other boards. Within the search, halving the key size, compressing the stored levels and streaming levels from disk would extend the reach to level 11; a corridor-aware search is the only route to orders of magnitude.

1. The puzzle and the rules the solver uses
2. The eight solves
3. The pattern in the move counts
4. What the solutions look like
5. How the Panex Solver works
6. What the search profiles show
7. Could the pattern make the solves faster?
8. Conclusions
9. Appendix: formulas, tables and methods
10. References

1. The puzzle and the rules the solver uses

Panex, invented by Toshio Akanuma and made by TRICKS Co. of Tokyo, looks like the Tower of Hanoi but behaves very differently. Three vertical tracks (columns) are joined at the top by a horizontal track, the bridge. Two towers of n pieces each (ten in the commercial puzzle) start on the two outer columns, one blue and one orange; the centre column is empty. The task is to exchange the two towers. A hidden tongue under each piece and a narrowing throat in each track enforce the one rule that makes the puzzle hard: **the piece that belongs on floor i of a tower can never go deeper than level i of any column.** The top floor is level 1 and the bridge is level 0, so the piece that belongs at the top of a tower can only ever sit in the top slot or in the bridge, while the bottom piece can go anywhere.

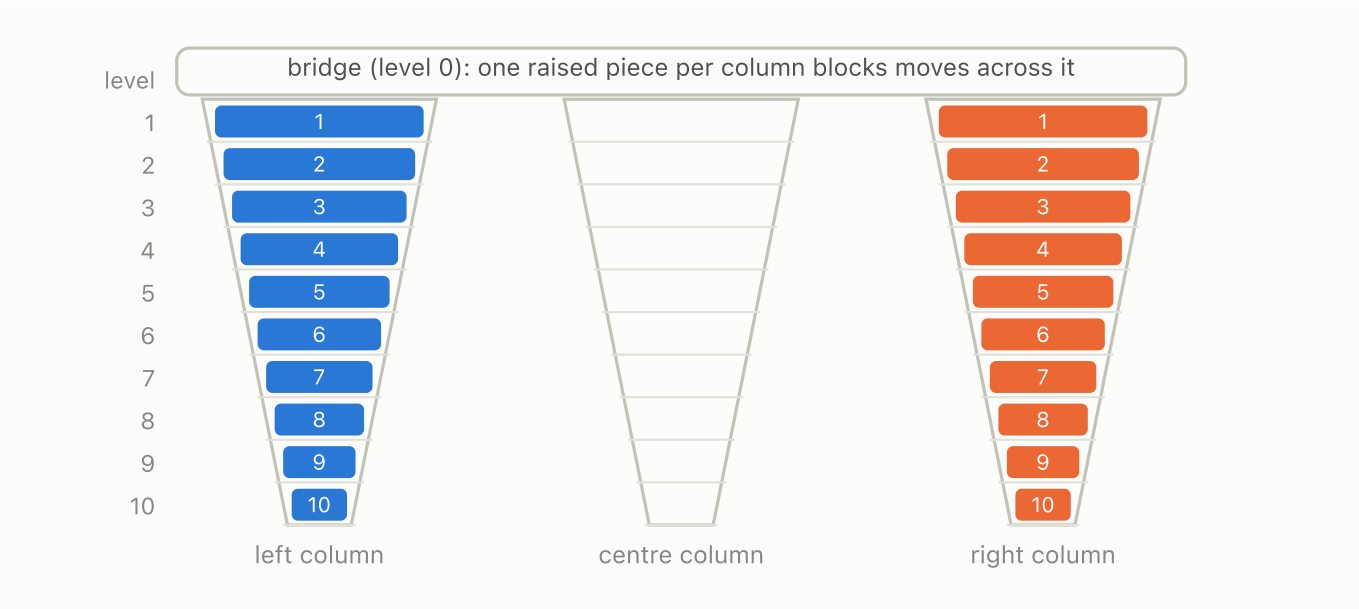


Figure 1. The level-10 board at the start. Each column has ten levels plus the bridge above it. A piece numbered i rests no deeper than level i, so a tower stands only in the order shown. The pieces slide along the tracks and cannot pass one another: a piece raised into the bridge blocks every move that would pass over its column.

Pieces slide, they are not lifted, so a piece cannot pass over another. A move takes the top piece of one column, up into the bridge, along it, and down another column; however far it travels, that counts as one move. A column holds at most n + 1 pieces: n in its levels and one more waiting in the bridge above them. A column whose top piece is in the bridge is *raised*: nothing can be put onto it, and nothing can travel across it along the bridge.

The solver, and this paper, use ShapeLogic's stacking model of the board, the Panex columns rule set: a disc of size s may sit at most s places below the top of its column, discs stack in any order, one disc more than the column height may wait raised in the bridge, and a raised disc blocks moves that pass over it. A stacking is simply the list of pieces on each column, bottom to top. The levels the pieces rest at are not part of the state: gravity settles each piece at its own limit or on the piece below, whichever is higher, and a column is raised exactly when some piece would otherwise sit below its limit. The model's raised test (some piece at index i from the bottom with limit smaller than count − i) is the same condition, so the reachable stackings and the legal moves are those of the physical puzzle. The eight problems in the file are the classic swap on three Panex columns of height n, with the left tower's pieces numbered 1 to n and the right tower's pieces of the same sizes.

Two tasks have been studied since the puzzle appeared: the **transfer** of one tower from a side column to the centre, whose minimum T(n) is known exactly, and the **swap** of the two towers, whose minimum X(n) is what the file's solves establish for levels 3 to 10.

2. The eight solves

Table 1 lists what the file records for each problem: the solution length, the stackings the search found, the search depth, and the time. The last three columns come from re-running levels 3 to 8 with the command-line solver on the same machine (an Apple M4 MacBook Pro with 32 GB, all cores, the level log on) to collect per-level data; the times agree with the file's within a few percent.

n	moves	stackings found	depth in file	time in file	rate	re-run time	re-run peak memory	re-run disk
3	42	1,924	42	25 ms	77K/s	0.001 s	0 MB	—
4	128	38,236	128	39 ms	980K/s	0.005 s	0 MB	—
5	343	738,490	342	118 ms	6.26M/s	0.077 s	5 MB	—
6	881	13,877,174	880	1.1 s	12.4M/s	1.062 s	111 MB	—
7	2,189	259,757,956	2,188	7.8 s	33.4M/s	7.822 s	2.1 GB	—
8	5,359	4,870,733,506	5,358	2.7 min	29.7M/s	168.212 s	4.3 GB	39 GB
9	13,023	91,449,852,580	13,022	49.9 min	30.5M/s	—	—	—
10	31,537	1,718,452,301,186	31,536	58.6 h	8.14M/s	—	—	—

Table 1. The solves in `Panex Swap 3–10.xmpuzzle`. "Stackings found" is the solver's own count: the stackings within half the solution length of the start, with each self-twin stacking counted twice (section 5.5). The depth in the file is one short for the odd solution lengths, because the search met its mirror image at the previous level before the depth counter was updated; the moves column is exact. The level-8 re-run kept every level on disk (39 GB) because the disk budget allowed it; the file's run on level 10 cannot have kept every level (section 6).

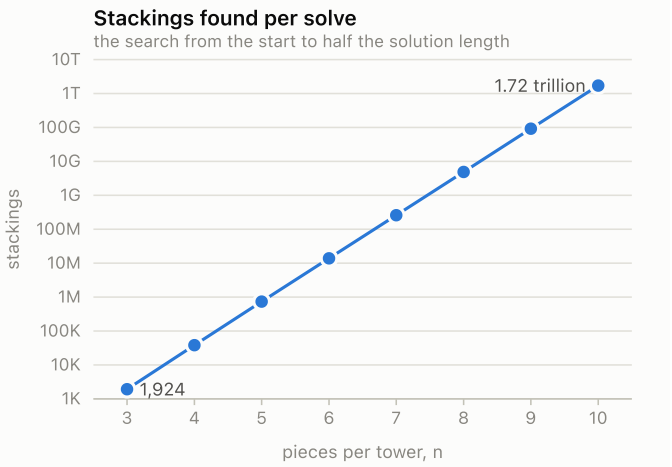


Figure 2a. Stackings found per solve. The count grows by a factor of about 18.8 per level.

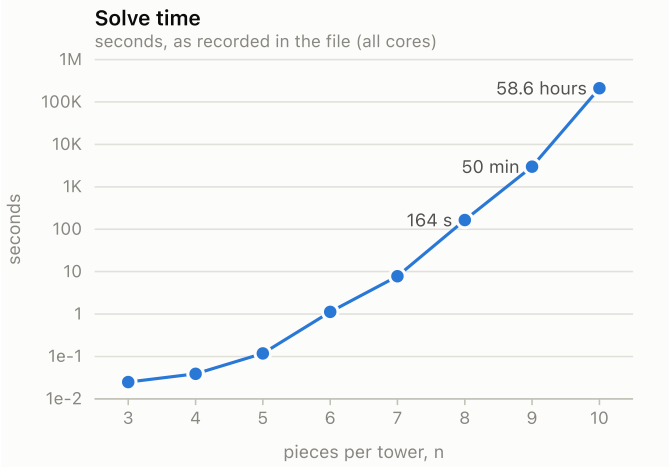


Figure 2b. Time per solve as recorded in the file. Up to level 9 each level costs about 18 times the previous one; level 10 took 70 times level 9.

Two things stand out. The solution grows by a factor of about 2.4 per level while the work grows by about 18.8 per level, so each level costs almost eight times more work per move of the answer. And level 10 ran at 8.1 million stackings per second where levels 7 to 9 ran at 30 to 33 million: three quarters of the 58.6 hours were spent slower than the search's natural pace. Section 6 discusses the likely causes.

3. The pattern in the move counts

3.1 The sequence and the silver ratio

The eight optimal lengths, with the published values for levels 1, 2 and 11 to 13, form the sequence 3, 13, 42, 128, 343, 881, 2189, 5359, 13023, 31537, 76245, 184191, 444807. The ratio between consecutive terms falls steadily: 4.33, 3.23, 3.05, 2.68, 2.57, 2.48, 2.45, 2.43, 2.42, 2.418, 2.416, 2.415, and the gaps to 2.4142 shrink by a little under half each time (0.0074, 0.0034, 0.0016, 0.0007). The limit is $1 + \sqrt{2}$, the silver ratio: the growth constant of the Pell numbers, just as the golden ratio is that of the Fibonacci numbers.

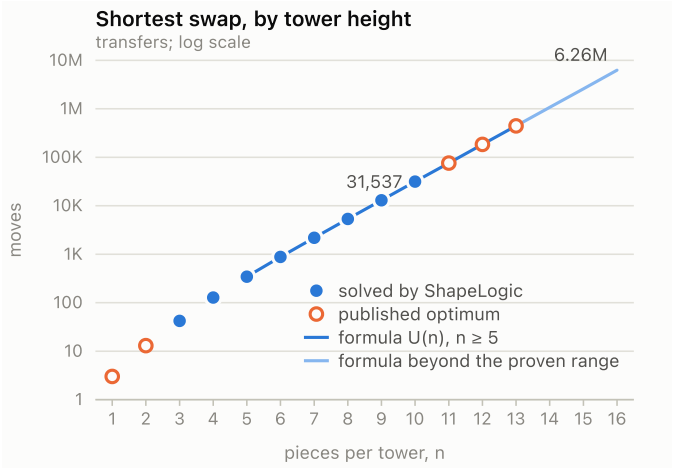


Figure 3a. The shortest swap by tower height, on a log scale: the eight values solved here, the five published ones, and the formula U(n) of section 3.2, which the data follow from level 5 on.

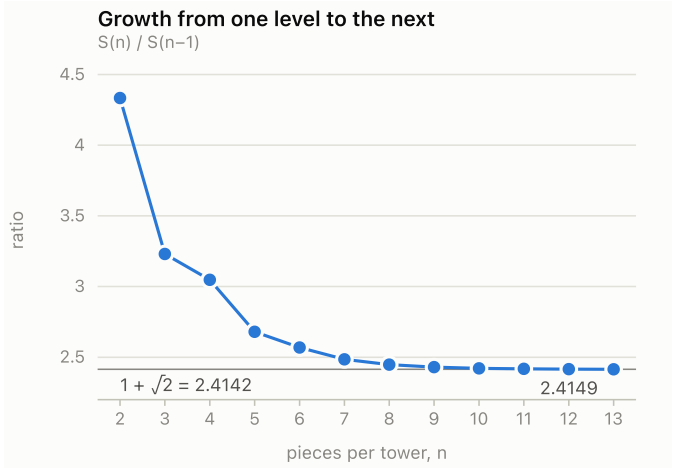


Figure 3b. The ratio of consecutive lengths closes on $1 + \sqrt{2}$ from above; the gap roughly halves at each level.

3.2 A recurrence and a closed form

A sequence that grows like $(1+\sqrt{2})^n$ suggests the Pell recurrence a(n) = 2a(n−1) + a(n−2). Applying it to the data leaves a residue that is exactly linear from level 7 on: X(n) − 2X(n−1) − X(n−2) = 84, 100, 116, 132, 148, 164, 180 for n = 7 … 13, which is 16n − 28 (Table 2). So

$$X(n) = 2X(n-1) + X(n-2) + 16n - 28 \quad (n \geq 7, \text{ with } X(5) = 343 \text{ and } X(6) = 881 \text{ as the base})$$

and, solving the recurrence, with P_k the Pell numbers 0, 1, 2, 5, 12, 29, 70, 169, 408, 985, 2378, …,

$$X(n) = 7\cdot(5P_{n-1} - P_{n-2}) - 8n - 2 = \tfrac{1}{2}\cdot 7\cdot(7 - 4\sqrt{2})\cdot(1+\sqrt{2})^n + \tfrac{1}{2}\cdot 7\cdot(7 + 4\sqrt{2})\cdot(1-\sqrt{2})^n - 8n - 2.$$

The formula was fitted on four values (levels 5 to 8) and then reproduces the five others that are proven (levels 9 to 13), including the three that Kisman computed with a different program. From level 6 on the $(1-\sqrt{2})^n$ term is below a quarter, so X(n) is the nearest integer to $4.70101\cdot(1+\sqrt{2})^n - 8n - 2$. Levels 1 to 4 do not fit: their solutions are shortened by special small cases, as the published analysis also notes. Table 2 gives the predictions for levels 14 to 16.

n	optimum X(n)	formula U(n)	X(n)/X(n−1)	X(n)−2X(n−1)−X(n−2)	16n − 28	source of X(n)
1	3	—	—	—	—	MSW
2	13	—	4.3333	—	—	MSW
3	42	—	3.2308	13	20	this file; MSW
4	128	—	3.0476	31	36	this file; MSW
5	343	343	2.6797	45	52	this file; MSW
6	881	881	2.5685	67	68	this file; MSW
7	2,189	2,189	2.4847	84	84	this file; Bagley 2002
8	5,359	5,359	2.4481	100	100	this file; Bagley 2002
9	13,023	13,023	2.4301	116	116	this file; Bagley 2008
10	31,537	31,537	2.4216	132	132	this file; Kisman 2010
11	76,245	76,245	2.4176	148	148	Kisman 2010
12	184,191	184,191	2.4158	164	164	Kisman 2010
13	444,807	444,807	2.4149	180	180	Kisman 2010
14	—	1,074,001	—	—	196	prediction
15	—	2,593,021	—	—	212	prediction
16	—	6,260,271	—	—	228	prediction

Table 2. The optimal swap lengths, the formula, the ratios and the residues of the Pell recurrence. Sources: MSW = Manasse, Sleator and Wei (1983, published 2004); Bagley's and Kisman's exhaustive searches as reported on Baxter's Panex page.

This is not a new formula. Manasse, Sleator and Wei's 1983 paper gives the number of moves of their swap algorithm, for n ≥ 5, as $U(n) = c_3(1+\sqrt{2})^n + c_4(1-\sqrt{2})^n - 8n - 2$ with $c_3 = (7/2)(7 - 4\sqrt{2})$ and $c_4 = (7/2)(7 + 4\sqrt{2})$, which is the expression above. What the file's solves add is independent confirmation that U(n) is the true minimum for every level from 3 to 10: the solver found no shorter path, and its search is exhaustive to half the solution length from each end. The paper also proves a lower bound $L(n) = 4\cdot(T(n) + T(n-1) - 2)$, which is 12.6% below U(n) at level 10 and 12.8% below in the limit (Table 3). Between 13 and infinity the exact value is therefore still open: the formula is the best known solution and the proven optimum only up to level 13.

3.3 Why 1 + √2: the transfer

The silver ratio enters through the transfer. Manasse, Sleator and Wei prove that any shortest transfer of an n-tower to the centre must first bring the n−1 upper pieces to the centre (a shortest T_{n-1} or its variant $\bar{T}_{n-1}$, in which the top piece ends in the opposite corner), must then sink piece n under them (S_{n-1}), make a fixed four-move exchange, and finish with a shortest T_{n-2}. The proof works with "unavoidable positions": configurations every shortest sequence has to pass through, found by looking at the moment a large piece moves for the first or last time. With $S(n) = \bar{T}(n) - 1$ the recurrence becomes

$$T(n) = 2\bar{T}(n-1) + T(n-2) + 3, \quad \bar{T}(n) = T(n) - [n \text{ even}], \quad \text{so} \quad T(n) = (7/4)\cdot Q_{n-1} + ((-1)^n - 2)/2,$$

where $Q_k = (1+\sqrt{2})^k + (1-\sqrt{2})^k$ are the companion Pell numbers 2, 2, 6, 14, 34, 82, 198, 478, 1154, 2786. The characteristic equation of $T(n) = 2T(n-1) + T(n-2)$ is $x^2 = 2x + 1$, whose root is $1 + \sqrt{2}$. The swap

algorithm is a composition of transfers, sinks and a few fixed macros, so its length satisfies the same kind of recurrence with the same dominant root, and the linear term −8n − 2 is the accumulated cost of the fixed macros, one set per level.

n	T(n) transfer	T̃(n) transfer, variant	X(n) swap	U(n) upper bound	L(n) lower bound	L/U
1	1	1	3	—	—	—
2	3	2	13	—	—	—
3	9	9	42	—	—	—
4	24	23	128	—	—	—
5	58	58	343	343	320	0.933
6	143	142	881	881	796	0.904
7	345	345	2,189	2,189	1,944	0.888
8	836	835	5,359	5,359	4,716	0.880
9	2,018	2,018	13,023	13,023	11,408	0.876
10	4,875	4,874	31,537	31,537	27,564	0.874

Table 3. The published transfer minimum T(n), its variant T̃(n), the swap minimum X(n), and the bounds of Manasse, Sleator and Wei. T(n) is proven optimal for all n; X(n) is proven only where it has been searched.

4. What the solutions look like

The file's solutions were decoded from the stored frames (three frames per transfer: lift, cross, drop) into lists of moves, one piece and two columns each, and examined for structure. Three regularities hold for all eight levels.

4.1 A transfer at each end

Until the deepest piece of a tower first moves, the other n−1 pieces of that tower are being moved to the centre column. In every solution found that takes exactly T(n−1) moves, the proven minimum of the transfer: 2018 moves at level 10, 835 at level 9, 345 at level 8, and so on (Table 4). After the deepest piece's last move, the same number of moves remain, and they are the mirror image of a transfer: the n−1 pieces of the other tower come down from the centre onto the deepest piece, which has just settled in its final column. The core in between, 87% of the solution at level 10, is where the two towers pass each other.

n	moves	moves before the deepest piece first moves	moves after its last move	T(n−1) / T̃(n−1)	core between them	core share
3	42	2	3	3 / 2	37	88.1%
4	128	9	9	9 / 9	110	85.9%
5	343	23	23	24 / 23	297	86.6%
6	881	58	58	58 / 58	765	86.8%
7	2,189	142	142	143 / 142	1,905	87.0%
8	5,359	345	345	345 / 345	4,669	87.1%
9	13,023	835	835	836 / 835	11,353	87.2%
10	31,537	2,018	2,018	2,018 / 2,018	27,501	87.2%

Table 4. Every solution begins and ends with an optimal transfer of the n−1 upper pieces. Only at level 3 are the two transfers of different lengths (2 and 3), where the variant T(2) = 2 is one move shorter than T(2) = 3.

4.2 The itinerary of the deepest pieces

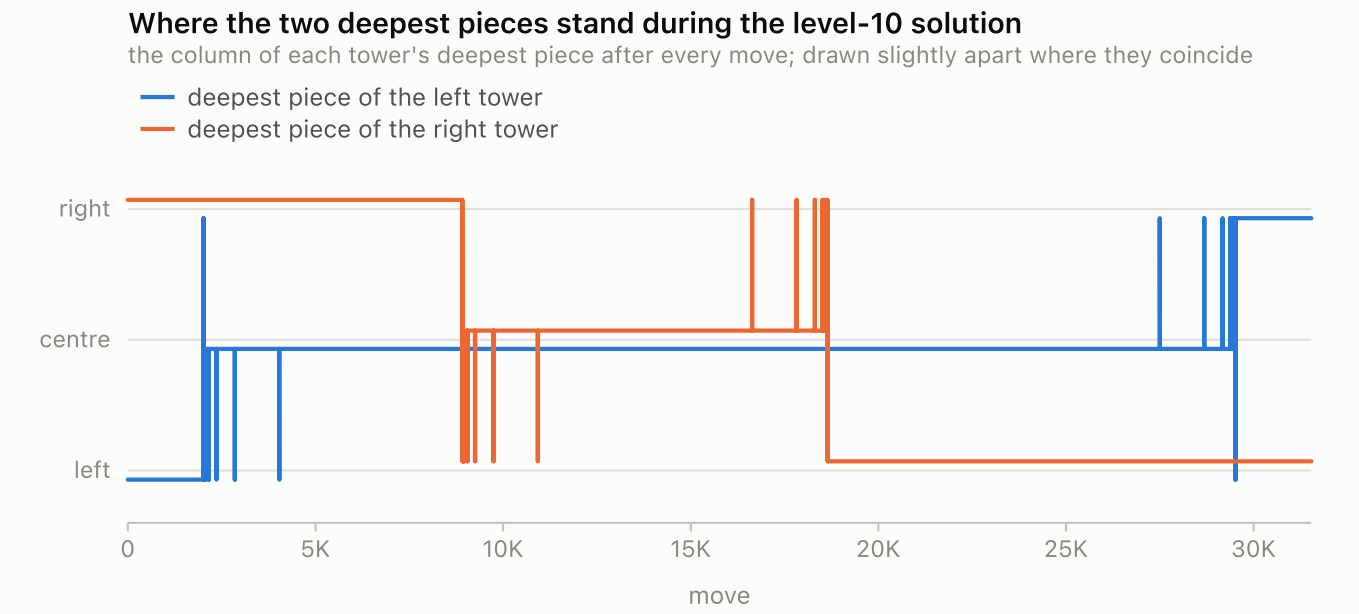


Figure 4. The column of each tower's deepest piece after every move of the level-10 solution. The left piece waits 2018 moves, then shuttles between the left and centre columns while the smaller pieces are rebuilt beneath it, reaches the bottom of the centre at move 4038 and stays there until move 27,500; the right piece makes its shuttle between moves 8910 and 10,929, sinks to the centre, and from move 16,637 climbs out into the left column, its goal, which it reaches at move 18,656. The left piece then crosses to the right column, settling at move 29,519, and the last 2018 moves bring the right tower's upper pieces down onto it.

The deepest pieces move rarely: 32 and 34 times at level 10, against 9,409 moves of the two size-1 pieces. Their itinerary is the skeleton of the solution. Jaap Scherphuis describes the same pattern for a human solver: bring the large blue piece to the centre, work it down to the bottom, then let it rise one level at a time as the next largest piece from the other side is sunk beneath it. The solver's optimal paths do exactly this. In the first shuttle (moves 2019 to 4038) the left deepest piece alternates between columns 3, 2 and 1 sixteen times while pieces 1 to 8 of its own tower are rebuilt under it on the left column, and ends alone at the bottom of the centre with piece 9 raised on the right. That phase brings the eight upper pieces back to the left column beneath the deepest piece, parks piece 9 raised on the right, and leaves the deepest piece alone at the bottom of the centre: 2020 moves, two more than the transfer before it.

4.3 Who does the moving

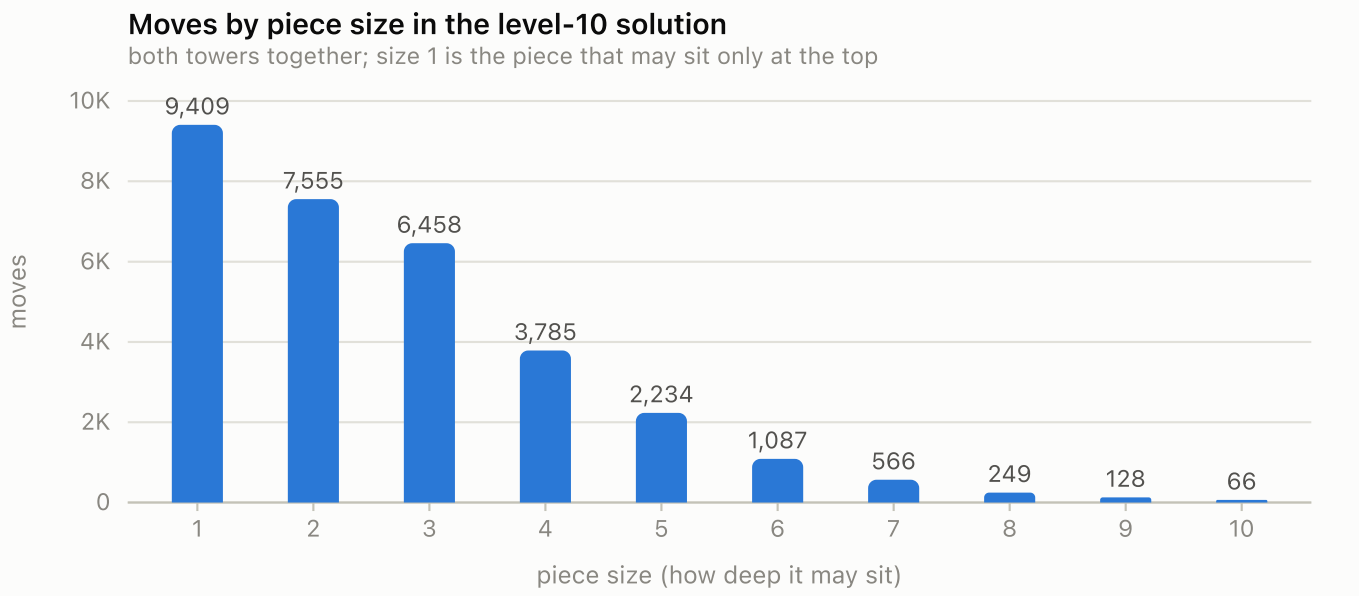


Figure 5. Moves by piece size in the level-10 solution, both towers together. The three shallowest sizes make 74% of all moves. Each deeper size moves less often than the one above: by a fifth or so at the top, by about half from size 4 down, until the two deepest pieces, which move only 32 and 34 times.

The shares are stable across levels: the size-1 pieces make 30 to 32% of the moves from level 6 on, size 2 about 24%, size 3 about 21%, size 4 about 12% (Table 8 in the appendix). The distribution is not unique, because the puzzle has an enormous number of shortest solutions (section 4.5) and the solver returns one of them; which tower's piece moves first is also a free choice, since the mirror image of a solution is a solution.

4.4 The midpoint

At the middle of each solution (move 15,768 of 31,537 at level 10) the two deepest pieces are the two bottom pieces of the centre column, the smallest left piece is alone on the left column, and the right column holds the other right pieces in order with a few small left pieces among them. This matches the "roughly half-way position" of the published algorithm, in which all pairs but the size-2 pair have changed sides and the two deepest pieces sit in the centre. The same shape appears at every level from 4 to 10, so it is a landmark that every optimal solution found passes through; it has not been proven that every optimal solution must.

4.5 How many shortest solutions there are, and where they run

For levels 3 to 5 the whole state space is small enough to search completely in Python (Table 7). Three facts come out of it. The space is a few times larger than what the solver visits: 1.9 million reachable stackings at level 5, of which the solver's half-depth search sees 39%. The goal is almost the farthest state from the start (343 moves away, while the farthest state is 378 away), which is why a one-directional search would have to visit nearly everything. And the number of distinct shortest solutions is astronomical, 6.3 trillion at level 5, yet they all run through a narrow corridor.

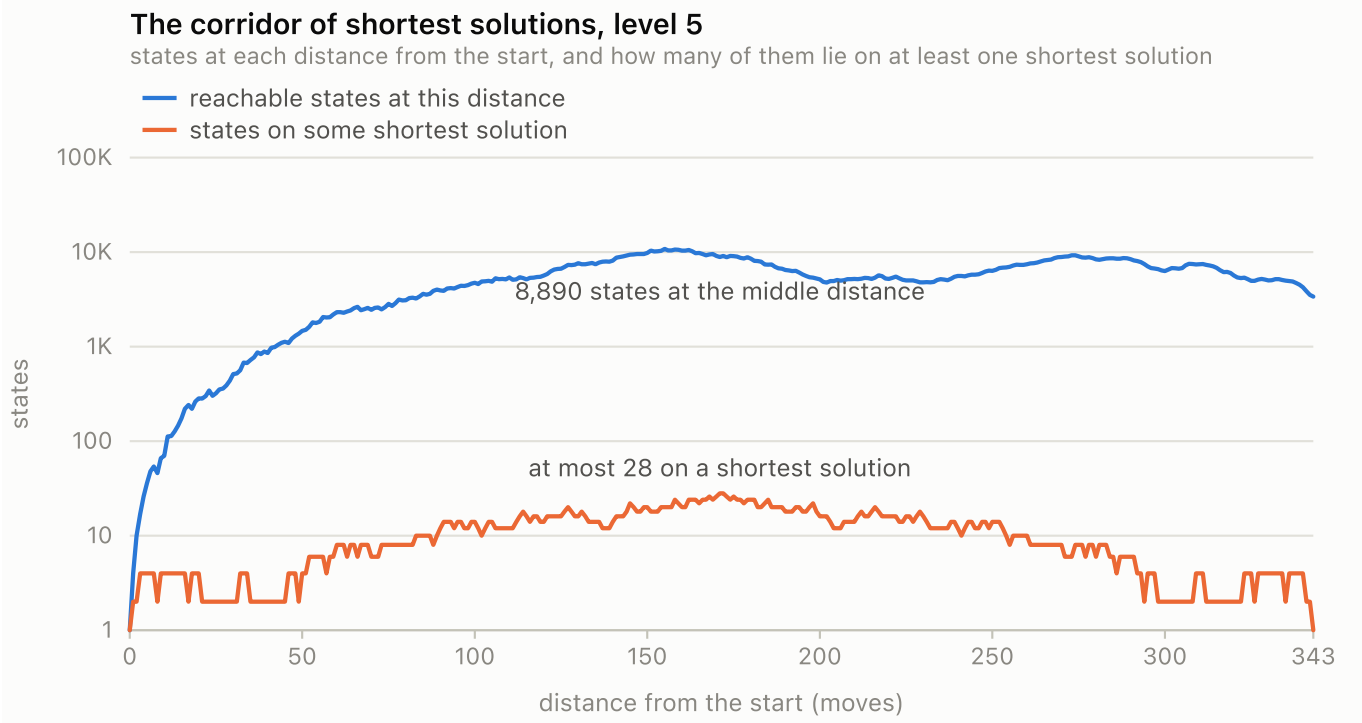


Figure 6. Level 5: the number of states at each distance from the start, and the number of those that lie on at least one shortest solution. The corridor never exceeds 28 states per level, against 8,890 states at the middle distance; its width is roughly symmetric about the middle, as the twin symmetry requires.

The corridor is where any structural shortcut would come from. A breadth-first search cannot know which of the 8,890 states at distance 171 are the 28 that matter; a method that could recognise them would do about 300 times less work at level 5, and the ratio grows quickly with the level, since the frontier grows 7.6 times per level while the corridor, judging by levels 3 to 5 (12, 8 and 28 states at its widest), grows slowly if at all. This is, in all likelihood, the kind of observation behind Kisman's search, which found the level-10 optimum in 30 minutes with 50 MB.

n	reachable stackings	farthest distance from the start	swap length	distinct shortest solutions	widest corridor level	states at the middle distance	self-twin states	states within half the length	stackings found by the solver
3	4,872	45	42	108	12	182	24	1,918	1,924
4	98,112	138	128	110,592	8	1,264	64	38,212	38,236
5	1,886,592	378	343	6,262,062,317,568	28	8,890	160	738,412	738,490

Table 7. Complete searches of the small levels. "States within half the length" is the ball the solver's mirrored search explores; the solver's count of stackings found is that ball plus the self-twin states within it, which it counts twice (and minus the start). "Self-twin" states are fixed by the twin symmetry of section 5.5.

4.6 What makes the puzzle hard: a relaxation experiment

If the two towers' pieces of some size were identical, so that either could end on either side, the swap could be shorter. Running the level-5 search with each choice of sizes made identical shows where the cost lives (Table 6). Making the two size-1 pieces identical saves nothing; the size-2 pair, one move; sizes 1 to 4 together, 23 moves of 343. Making the deepest pair identical saves 215 moves and leaves exactly X(4) = 128, the swap of the four pieces above them. The same holds at levels 3 and 4 (the deepest pair identical gives X(2) = 13 and X(3) = 42). The cost is entirely in getting the deepest pieces past each other

with everything else in the way; the identities of the small pieces are nearly free, even though they do most of the moving.

pieces of these sizes made identical across the two towers	shortest swap	moves saved
none	343	0
size 1	343	0
size 2	342	1
size 3	339	4
size 4	326	17
size 5 (the deepest)	128	215
sizes 1 and 2	342	1
sizes 1, 2, 3 and 4	320	23
sizes 4 and 5	42	301
sizes 2, 3, 4 and 5	3	340
all five	0	343

Table 6. Level 5 with pieces of the given sizes treated as interchangeable between the towers: the shortest path from the start to any of the resulting goal positions.

5. How the Panex Solver works

The Panex Solver is ShapeLogic's search for the fewest rod transfers on stacking puzzles with Panex columns; the Stacking Solver runs on it too whenever the puzzle fits its keys. It is a breadth-first search over stackings, level by level from both ends, that keeps only the newest levels in memory, writes every few levels to disk when they do not fit, traces the path back between them once the ends meet, and can stop and carry on later. It was ported line for line from the original ShapeLogic implementation (`panex.cpp`), with its parallel steps moved onto a thread pool; the saved search files are byte for byte the original's, so a search stopped by one program carries on in the other.

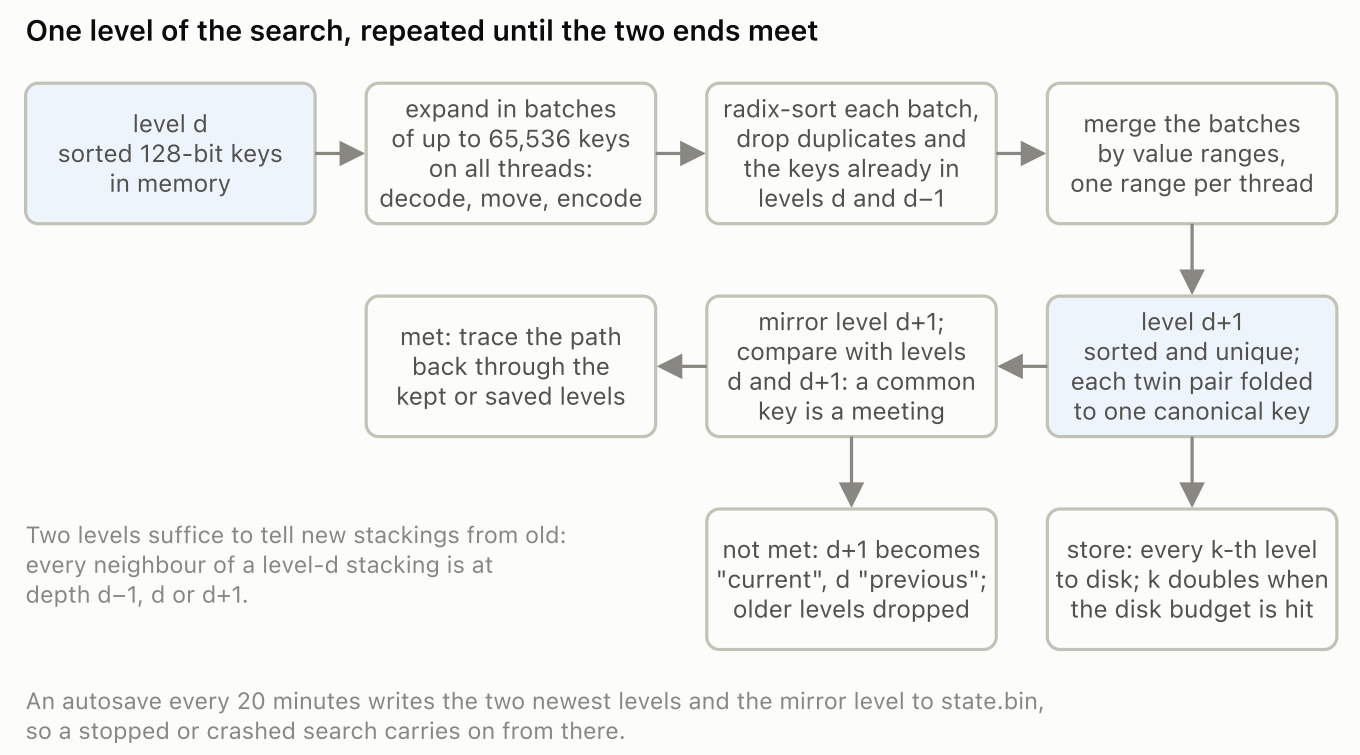


Figure 7. One level of the search. The search runs from the start only, because the goal of a swap is the mirror image of the start: the frontier at depth d from the goal is the mirror of the frontier at depth d from the start, so comparing a level with the mirror of the previous and current level is a meeting of two searches without running the second.

5.1 A stacking as a 128-bit key

A stacking is encoded in five bits per symbol: the pieces are 0 to 30, and 31 separates one column from the next. The three columns' pieces, bottom to top with a separator between columns, are packed into two 64-bit words, high word first, so that the keys sort as integers and a level is simply a sorted array of 16-byte keys. The code allows 25 symbols (31 discs on up to 8 rods, with the sum of discs and rods at most 26); on three columns that is 23 pieces, so the largest swap the solver can represent is level 11. Decoding a key back to the three stacks, generating its moves under the rules (depth limits, the raised test, blocking across the bridge, column capacity, and optionally a pocket column, an outside channel, the size rule of plain rods or the distance rule) and re-encoding the results is the inner loop. The rules object lays everything out in small fixed arrays so this loop touches no heap memory.

5.2 Levels, and why two levels are enough to find what is new

The search keeps the current level and the previous one. Each new stacking generated from the current level is looked up in both; what is in neither is new and forms the next level. This is correct because every move is reversible, so a neighbour of a stacking at distance d is at distance d−1, d or d+1 and nothing older can reappear. Not keeping a visited set is what makes the memory of the search proportional to the width of the frontier rather than to the number of stackings visited: at level 10 the widest level holds perhaps 130 million keys (2 GB), while the search visits 860 billion canonical stackings (13.8 TB).

5.3 Batches, sorting and merging on every core

The current level is cut into batches of at most 65,536 keys, which the pool's threads take from a counter so that no thread waits on another. A thread decodes each key of its batch, generates the moves, encodes them, sorts the batch with an 11-bit-digit radix sort (several times faster than a comparison sort on these keys), drops duplicates, and subtracts the current and previous levels. The subtraction walks the sorted batch and the sorted level together, searching each key from where the last one was found in

doubling steps and then by halving, a galloping search that stays within a few cache lines when the keys are close, as they are. The batches, each sorted and unique, are then merged into the next level: the key range is cut into as many stretches as there are threads, by sampling, and each thread merges one stretch of every batch, so the final merge is not left to one thread as the last round of a pairwise merge would be. A batch that would push the level over the memory budget abandons the level, which ends the search with a saved state rather than an out-of-memory crash. Small levels use fewer threads (one per 20,000 keys), because starting a thread costs more than a small level. Whatever the thread count, the path found is the same.

5.4 Meeting in the middle, in a mirror

A search from both ends finds a shortest path after exploring about two balls of half the radius instead of one ball of the full radius, which for a graph whose levels grow exponentially is the square root of the work. When the goal is the mirror image of the start, as in every swap, the solver runs only the forward search and compares each new level with the mirror images of the previous and the current level. A key in common is a meeting: a stacking at distance d from the start whose mirror image is at distance d or d−1 from the start, hence at that distance from the goal. The solution length is then 2d or 2d−1. For a puzzle whose goal is not a mirror image (a pocket column, or a different arrangement) the solver runs both ends and expands whichever frontier is smaller.

5.5 Twin folding

The swap has one more symmetry. Mirroring the board and at the same time exchanging each piece with the other tower's piece of the same size maps the start to itself and the goal to itself. Two stackings related by this map, twins, are at the same distance from the start, so the search keeps only the smaller key of each pair (the canonical key) and every level is half as large. The solver checks at the outset whether such a permutation of the pieces exists (`findSigma`): it must pair pieces of equal depth limit and fix both ends. This is why the solver's count of stackings found is twice the number of canonical keys, and why the self-twin stackings, which are their own canonical key, are counted twice (Table 7).

5.6 Memory, disk, and tracing the path back

To recover the path after the ends meet, the search needs its old levels. While they fit in a quarter of the memory budget they are kept in memory; after that they are written to disk, in the search's own folder, and the search goes on from the newest two levels. Each level is written by the first job of the next level's expansion, so writing overlaps the search (the original used a separate writer thread). The disk budget is half the free space, at most 200 GB; when the stored levels exceed it, the interval between stored levels doubles and every other file is deleted, so the store holds levels 0, k, 2k, … Tracing the path back then works between stored levels: from the meeting stacking, the solver grows a ball of neighbours until it reaches the previous stored level, finds the stacking there that the ball touched, and walks the ball back; and so on to the start. The mirror half of the path is traced through the same forward levels in the mirror. When the search has no folder at all it still solves: once the levels no longer fit, it solves the two halves of the path from the start to the meeting point and from there to the goal, recursively, each a smaller search.

The memory budget is half the machine's memory by default (the engine's Search memory setting), of which a quarter goes to kept levels, the rest to the two current levels, the mirror level and the batches. A level that would not fit ends the search with its state saved.

5.7 Saving, resuming and the rest

Every 20 minutes (and when it is stopped, or runs out of memory) the search writes the two newest levels, the mirror level, the depths, the count of stackings and the time used to `state.bin`, named by a

fingerprint of the rules, the start and the goal, so that a stopped search carries on exactly where it was, in the app or in the command-line solver, and the stored levels on disk let it trace the path once it meets. A damaged or crafted state file is refused rather than trusted (each layer's count is checked against the file's size). The classic Tower of Hanoi (plain rods, the size rule, one tower moved) is answered by its $2^n - 1$ recursion without a search. A level log can be switched on (`--panex-log`) to see each level's size, which is how the profiles in section 6 were collected.

5.8 What has been done for speed

- The frontier-only memory of the search (two levels and a canonical key per twin pair) is what makes level 10 fit in 16 GB of memory at all; a conventional visited set of 860 billion keys could not.
- The mirror meeting halves the depth and the twin folding halves the width: together they cut the stackings generated by a factor of about 2 × 2 × (the frontier's growth over the second half of the depth), which at level 10 means the search explores 1.7 trillion rather than many tens of trillions of stackings.
- All of the per-level work runs on every core: the expansion in batches, the radix sorts, the subtraction, the range-partitioned merge, the mirror level's computation and sort. On the level-7 swap the port runs within 2% of the original with all threads and 20% faster on one thread, because the writer's work became a job in the pool rather than a thread the pool could not spare.
- The galloping subtraction keeps the lookups of new keys in the current and previous levels local in memory; it also fixed undefined pointer arithmetic that the original had and that the sanitiser build caught.
- The 11-bit radix sort of each batch and the 16-byte keys keep the sort cost at a few passes over memory; the merge by value ranges keeps every thread busy to the end of a level.
- Levels are written to disk while the next level is computed, and the disk store thins itself rather than failing when the budget is reached.

---

6. What the search profiles show

The level logs of the re-runs give the width of the search at every depth for levels 3 to 8 (Figure 8, Table 5). The shape is the same at every level: the frontier grows exponentially at first, by about 2.4 per move early on, then more slowly, and reaches its widest point at about 88% of the half length, just before the two ends meet. The widest level grows by 7.6 per level of the puzzle, the total by 18.8, the last level by 7.7. The ratios are steady enough to extrapolate: the level-10 search had a widest level of roughly 130 million keys (2 GB) and 860 billion canonical keys in all (13.8 TB at 16 bytes each), and a level-11 search would have a widest level near a billion keys, about 16 GB, with 16 trillion keys in all. Three such levels plus the mirror level must be in memory at once, so level 11 is out of reach of a 32 GB machine with the current design, and at the level-9 rate of 30 million stackings a second it would take about two weeks of search.

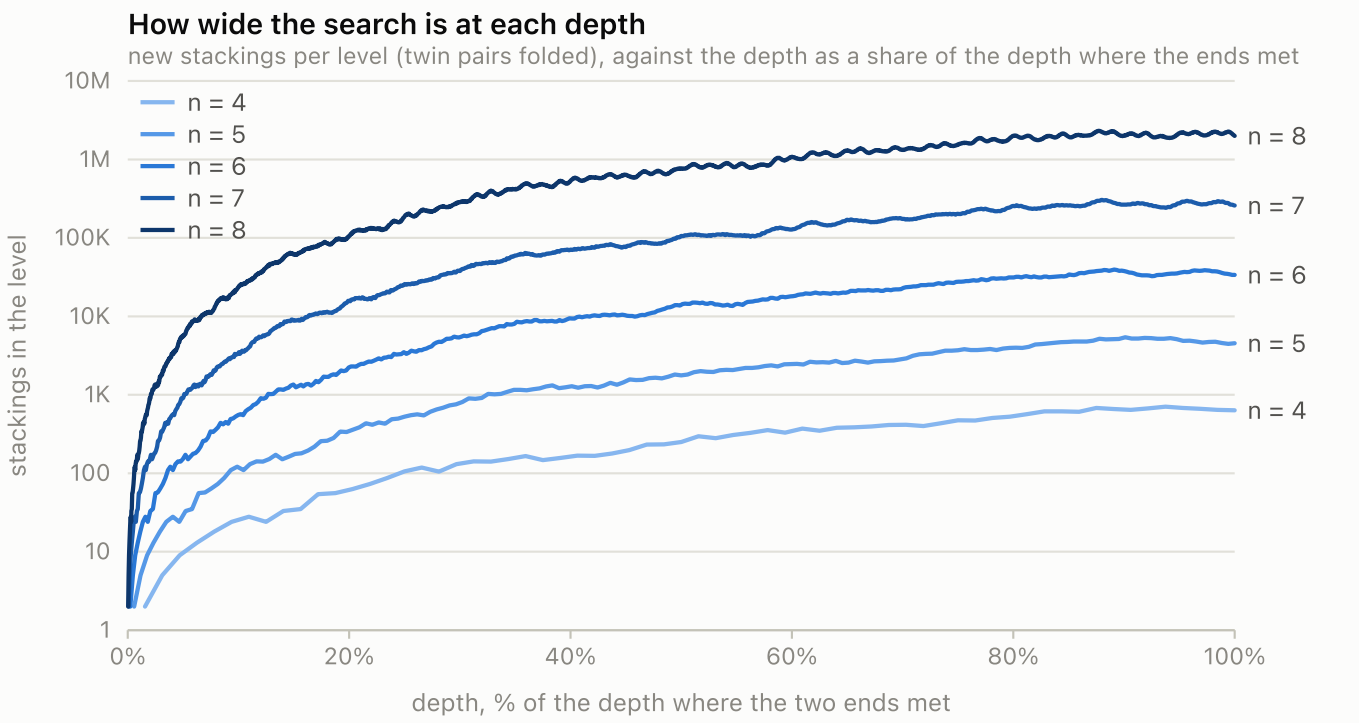


Figure 8. The width of each search level, for levels 4 to 8 of the puzzle, against the depth as a share of the depth where the two ends met. Level 3, with only 21 levels, is omitted. The curves have the same shape and are spaced by a factor of about 7.6.

n	levels searched	widest level (keys)	where	last level	keys in all levels	memory of the widest level	disk for every level
3	21	92	95%	91	962	0.0 MB	0.00 GB
4	64	704	94%	633	19,118	0.0 MB	0.00 GB
5	172	5,411	90%	4,548	369,245	0.1 MB	0.01 GB
6	441	39,340	89%	33,691	6,938,587	0.6 MB	0.11 GB
7	1095	304,105	88%	257,923	129,878,978	4.9 MB	2.08 GB
8	2680	2,310,894	88%	1,992,506	2,435,366,753	37.0 MB	38.97 GB
9 (est.)	6,512	~17.6M	~88%	—	~45.8G	~0.3 GB	~0.7 TB
10 (est.)	15,769	~133M	~88%	—	~861G	~2.1 GB	~13.8 TB
11 (est.)	38,123	~1.01G	~88%	—	~16.2T	~16.2 GB	~258.9 TB

Table 5. The level profiles from the re-runs, and extrapolations for levels 9 to 11 at the measured ratios of 7.6 (widest level) and 18.8 (total). Keys are canonical: each stands for a twin pair. "Disk for every level" is the space needed to keep every level for tracing the path; beyond the disk budget the store thins to every 2nd, 4th, … level. Sizes are decimal (GB = 10^9 bytes).

6.1 The level-10 slowdown

Levels 7, 8 and 9 ran at 33, 30 and 31 million stackings per second; level 10 averaged 8.1 million, so about 44 of its 58.6 hours were spent beyond what its size predicts. The file does not record why, but the data narrow the possibilities. The widest level (about 2 GB) and the two or three levels in memory at once fit the 16 GB budget, so the search itself was not memory bound, but the application and the rest of the system share the 32 GB, and macOS compresses or swaps under pressure. The store cannot have kept every level: 13.8 TB of levels against a budget of at most 200 GB means the interval doubled to at least 128, so tracing the path back grew balls of up to 127 moves between stored levels, each a search of its

own; the autosave wrote the three newest levels, up to about 6 GB, every 20 minutes for 58 hours, several hundred gigabytes of writes in all; and a laptop running 58 hours at full load may throttle. A re-run with the level log on, which prints the time per level, would show where the hours went; section 7.4 lists what to change if the trace-back or the writes are to blame.

---

7. Could the pattern make the solves faster?

7.1 What is certain, and what is not

Three things are proven. The transfer minimum T(n) is known for every n, with a proof that works for all n. The swap minimum X(n) is known for n ≤ 13 by exhaustive search, and for n = 3 to 10 the file's solves are an independent proof: the solver's search is complete to half the length from each end, so no shorter path exists. The formula of section 3.2 is the length of a known algorithm, hence an upper bound for every n; the proven lower bound is 12.8% below it. So for n ≥ 14 the formula is a prediction, not a certainty, and the only known ways to settle a level are an exhaustive search of some kind or a new proof. Nothing in the eight solves contradicts the formula, and the regularity of the structure found (section 4) is evidence that the algorithm is optimal in general, but evidence is not proof.

7.2 Construct rather than search

For the standard puzzle, the answer need not be searched for at all. The transfer algorithm is fully specified in the published paper ($T_n = T_{n-1}$, S_{n-1}, four moves, T_{n-2}, with the small cases given), and the swap algorithm is specified as four sequences of these macros plus a few fixed move sequences (the paper's Y, W, V and the four-move exchange), which the solutions found here match phase by phase. Implementing the construction would answer any standard swap or transfer in the time it takes to write the moves out, a few milliseconds for level 10 against 58.6 hours, with a length proven optimal to level 13 and the best known beyond. This is the same kind of shortcut the solver already takes for the Tower of Hanoi. It would be honest to flag such a result as "known optimal (proven to level 13)" rather than "shortest found by search", and to keep the search for everything else: pocket columns, the outside channel, mixed piece sizes, other start and goal positions, and for anyone who wants the proof.

7.3 Using the structure inside a search

The regularities of section 4 could shorten a proof search, but less than one might hope, because the cost of a breadth-first search is in its widest levels and the regularities constrain the ends. Fixing the first and last T(n–1) moves (2018 of 15,769 levels per side at level 10) removes the cheapest 13% of the levels. Splitting the search at the midpoint landmark would need a proof that every optimal solution passes through it, and even then the two half searches would each reach a frontier about a third as wide as the current peak, a saving of perhaps three to five times. Lower-bound heuristics, the usual way to prune such searches, are weak here: the relaxation experiment shows that dropping the small pieces from the problem keeps almost all of the length but leaves a state space that is still astronomically large, while dropping the deepest pieces loses the length entirely. The orders of magnitude lie in the corridor (section 4.5): a search that generates only the stackings that can be on a shortest path, which is what the unavoidable-position arguments of the published proof do for the transfer, and what Kisman's 30-minute search presumably does for the swap. That is a research project rather than a tuning, and the first step would be to obtain and study his method.

7.4 Engineering steps within the current design

- **Half the key size.** The 128-bit key spends 120 bits on 24 five-bit symbols; the reachable stackings of the level-10 swap number about 5×10^{12} (Table 7's counts grow 19-fold per level), which a ranking of the arrangement would fit into 43 bits. Eight-byte keys halve the memory of every level, the disk, and the bytes every sort and merge moves. The format of saved searches would change, so the original's files would no longer be shared.
- **Compress the stored levels.** Sorted keys delta-encode well: the gaps between neighbours in a level are small, so a variable-length encoding of the differences should take a few bytes per key instead of 16. That lets the store keep every 16th or 32nd level where it now keeps every 128th at level 10, which shortens the trace-back, and it cuts the terabyte of autosave writes to a fraction.
- **Stream the levels.** Level 11 needs about 16 GB per level. A search that keeps the current and previous levels on disk and streams them through the expansion (an external-memory breadth-first search, which the sorted-run structure of the levels already suits) would make level 11 a question of weeks of disk time rather than of memory.
- **Find the level-10 hours.** Run level 10 with the level log and the per-level times, or add the timing of the trace-back and the saves to the solver's progress, before changing anything else; the fix depends on which of the candidates in section 6.1 it is.
- **Count exactly.** Count self-twin stackings once in the stackings-found figure and record the exact depth for odd lengths, so that the file's statistics match the ball sizes in Table 7.

---

8. Conclusions

The eight solves settle, independently of the published searches, the shortest swap for towers of 3 to 10 pieces, and they fit a closed formula built on the Pell numbers: the move count of the Manasse, Sleator and Wei algorithm, which grows by the silver ratio $1 + \sqrt{2}$ per level. The solutions share one skeleton: an optimal transfer at each end, the two deepest pieces shuttling and sinking in the centre in between, the smallest pieces doing three quarters of the work, and a recognisable midpoint. The solver that found them is a carefully engineered frontier search: 128-bit keys, two levels in memory, a mirror in place of a second search, twin folding, radix sorts and range-partitioned merges on every core, and a disk store that lets it pause, resume and trace its path. Its cost grows 18.8-fold per level against a 2.4-fold answer, so level 10 is close to the limit of a 32 GB laptop and level 11 is out of reach without streaming levels from disk. The clearest gain is not in the search at all: for the standard swap the answer can be constructed from the known algorithm in milliseconds, with optimality proven to level 13. Beyond that, the measured corridor of shortest solutions, a few dozen states wide against frontiers of millions, is where a structural search would find its orders of magnitude; it is also the open mathematical question, since the formula is proven only as far as the searches have gone.

---

9. Appendix: formulas, tables and methods

A.1 Formulas

$$\text{Pell numbers: } P_0 = 0,\ P_1 = 1,\ P_n = 2P_{n-1} + P_{n-2}. \quad \text{Companion Pell numbers: } Q_n = (1+\sqrt{2})^n + (1-\sqrt{2})^n = 2(P_n + P_{n-1}).$$

$$\text{Transfer: } T(n) = (7/4)\cdot Q_{n-1} + ((-1)^n - 2)/2 \text{ for } n \geq 3; \quad \bar{T}(n) = (7/4)\cdot Q_{n-1} - 3/2; \quad S(n) = \bar{T}(n) - 1.$$

$$\text{Swap: } X(n) = 7\cdot(5P_{n-1} - P_{n-2}) - 8n - 2 = 7\cdot(7P_{n-1} - P_n) - 8n - 2 \text{ for } 5 \leq n \leq 13 \text{ (proven)},$$
$$\text{conjectured beyond; } \quad X(n) = 2X(n-1) + X(n-2) + 16n - 28 \text{ for } n \geq 7.$$

$$\text{Lower bound: } L(n) = 4\cdot(T(n) + T(n-1) - 2); \quad L(n)/U(n) \to (12 + 2\sqrt{2})/17 \approx 0.872.$$

Checks: X(10) = 7·(5·985 – 408) – 82 = 31,537; X(13) = 7·(5·13860 – 5741) – 106 = 444,807; T(10) = (7/4)·2786 – 1/2 = 4875.

A.2 Moves by size at every level

n	size 1	size 2	size 3	size 4	size 5	size 6	size 7	size 8	size 9	size 10
3	17 (40%)	15 (36%)	10 (24%)	—	—	—	—	—	—	—
4	44 (34%)	37 (29%)	29 (23%)	18 (14%)	—	—	—	—	—	—
5	111 (32%)	87 (25%)	72 (21%)	45 (13%)	28 (8%)	—	—	—	—	—
6	274 (31%)	218 (25%)	185 (21%)	106 (12%)	64 (7%)	34 (4%)	—	—	—	—
7	665 (30%)	533 (24%)	454 (21%)	268 (12%)	148 (7%)	77 (4%)	44 (2%)	—	—	—
8	1,592 (30%)	1,331 (25%)	1,123 (21%)	614 (11%)	382 (7%)	172 (3%)	96 (2%)	49 (1%)	—	—
9	3,853 (30%)	3,146 (24%)	2,714 (21%)	1,551 (12%)	920 (7%)	453 (3%)	221 (2%)	105 (1%)	60 (0%)	—
10	9,409 (30%)	7,555 (24%)	6,458 (20%)	3,785 (12%)	2,234 (7%)	1,087 (3%)	566 (2%)	249 (1%)	128 (0%)	66 (0%)

Table 8. Moves made by the pieces of each size (both towers together) in the file's solutions, with their share of the solution.

A.3 Methods

- Each solution's frames in the solve file (three per transfer) were turned into resting stackings and then into transfers, checking that exactly one piece changes column per transfer, from and to the top of a column.
- Levels 3 to 8 were re-solved with `shapelogic_cli -d --solver panex --panex-log --high-memory` to obtain the size of every level; the solver deletes its folder when it finishes.
- Levels 3 to 5 were searched completely in Python with the same rules, counting states per distance, shortest solutions (dynamic programming over the breadth-first layers), states on some shortest path (distance from the start plus distance to the goal equal to the length), self-twin states, and the relaxed optima of section 4.6 (the shortest distance from the start to any goal with the chosen pairs exchanged).
- The recurrence was found from the residues of the Pell recurrence and checked against the nine proven values; the closed form was then identified with the published U(n).

10. References

1. M. Manasse, D. Sleator, V. K. Wei, N. Baxter, "The Panex Puzzle", in *A Tribute to a Mathemagician*, A K Peters, 2004 (based on the unpublished 1983 paper "Some results on the Panex puzzle"). puzzleworld.org/SlidingBlockPuzzles/panex/panex5.pdf
2. N. Baxter, "Panex Puzzle Resources": the papers, the original search program, the news of D. Kisman's 2010 verification of 31,537 moves and his results for levels 11 to 13. baxterweb.com/puzzles/panex/
3. N. Baxter, "Recent Results for the Panex Puzzle", G4G5, 2002, revised 2008 (D. Bagley's confirmation of level 9).
4. J. Scherphuis, "Panex", Jaap's Puzzle Page: the recursion, the move-count tables and the human solving strategy. jaapsch.net/puzzles/panex.htm
5. R. Stegmann, "Sequential Movement Puzzles", Rob's Puzzle Page (the date of Kisman's result, 21 November 2010). robspuzzlepage.com/seqmove.htm
6. ShapeLogic sources: `core/shapelogic/solve/panex_search.{h,cpp}`, `core/shapelogic/model/stacking.{h,cpp}`, `docs/handoff/deferred-3a.md`, decision 0015.